

Comparing a Tuned Random Forest and a Neural Network for Predicting Horsepower Category in Used Cars

Osamuyimen Samson Osunde

Student Number: 20093878

B9DA109 – Machine Learning and Pattern Recognition

CA2, Task 2 – Individual Report

Abstract

This report compares two supervised learning approaches for predicting horsepower category (Low, Medium, High) in a 201-vehicle automobile dataset: a tuned Random Forest classifier and a feed-forward artificial neural network (ANN). Both models were trained on the same leakage-free, SMOTE-balanced feature set and evaluated on an identical held-out test split. After hyperparameter tuning, the Random Forest achieved 97.6% test accuracy and a macro F1-score of 0.95, narrowly outperforming the tuned ANN, which reached 95.1% accuracy and a macro F1-score of 0.93. Both models handled the class imbalance present in the raw data reasonably well once SMOTE oversampling was applied, though the Random Forest proved marginally more robust on the minority High-horsepower class. The report discusses the data preparation pipeline, the tuning process for each model, the resulting metrics, and the practical trade-offs between the two approaches.

1. Introduction

Predicting a vehicle's horsepower category from its other specifications is a reasonably contained tabular classification problem, but it is a useful one to explore two very different machine learning paradigms side by side: an ensemble tree-based method and a neural network. This report is based on Task 1 of the B9DA109 Machine Learning and Pattern Recognition module assessment (Abdel-Hak, 2026), in which my group built and compared a Random Forest classifier and an artificial neural network on a 201-row automobile dataset, originally compiled from 1985 Ward's Automotive Yearbook data (Schlimmer, 1987). My task here is to evaluate the two models individually, focusing on the data preparation choices, the hyperparameter tuning that was applied, the resulting performance, and what the results suggest about when each type of model is the more sensible choice.

2. Data Preparation

The dataset contains 201 cars described by 29 columns covering dimensions, engine specifications, fuel economy and price, alongside a horsepower-binned column that groups cars into three horsepower categories: Low, Medium and High. Two administrative index columns left over from a previous export were dropped immediately, as they carried no predictive value.

Two gaps needed addressing before modelling could begin. Four rows were missing a value for stroke, a continuous engine measurement, and these were filled with the column mean, since the missing proportion was small (under 2%) and the distribution showed no strong skew. One row had no value in the target column itself; rather than dropping this row, it was imputed with the modal class (Low). In hindsight this is a debatable choice for a target variable, since filling in an answer rather than discarding the row it belongs to risks introducing a label that was never actually observed, and a more defensible approach in any future iteration of this work would be to drop that row instead.

A more important decision was to exclude the raw horsepower column from the feature set entirely. horsepower-binned is calculated directly from horsepower, so leaving the raw value in as a predictor would have let both models solve the task almost trivially, by reading the near-exact answer off a single column rather than learning from the vehicle's other characteristics. This is a textbook case of data leakage, and removing it was necessary for the results reported here to mean anything.

The class distribution itself is uneven: 116 cars fall into the Low category, 62 into Medium, and only 23 into High. This imbalance matters, because a model could reach a deceptively high accuracy simply by predicting Low most of the time while still failing badly on the minority classes. To address this, the training data (after an 80/20 train-test split, giving 160 training rows and 41 test rows) was rebalanced using SMOTE, the Synthetic Minority Over-sampling Technique (Chawla et al., 2002), which generates synthetic examples of the minority classes rather than simply duplicating existing ones. Before SMOTE, the training split contained 18 High, 92 Low and 50 Medium examples; after SMOTE, all three classes were represented by 92 examples each. SMOTE was applied only to the training data, since the test set needed to be left in its original, imbalanced state to reflect the real-world distribution the models would eventually be judged against.

Numeric features (18 columns, including engine size, weight, dimensions and price) were standardised to zero mean and unit variance using scikit-learn (Pedregosa et al., 2011), which matters particularly for the neural network, where features on very different scales can otherwise dominate the learning process. The nine categorical columns (make, body style, drive wheels, and so on) were one-hot encoded. Both transformations were fitted on the training data only and then applied to the test data, so that no information from the test set could leak into preprocessing. After encoding, the feature space expanded from 27 raw columns to 72 numeric columns.

3. Hyperparameter Tuning

The Random Forest (Breiman, 2001) was first trained with close to default settings as a baseline, reaching 97.6% test accuracy, then tuned using GridSearchCV with 5-fold cross-validation over a small grid of tree counts (50, 100, 150, 200 and 250 estimators). The search selected 100 estimators as the best-performing configuration, with a cross-validated accuracy of 96.4%.

The ANN's architecture and training settings were chosen by comparing three candidate configurations on a validation split carved out of the SMOTE-balanced training data: a small network (16 and 8 units across its two hidden layers, no dropout, learning rate 0.001), a medium network (31 and 14 units, 20% dropout, learning rate 0.001), and a larger network (64 and 32 units, 30% dropout, learning rate 0.0005). Dropout (Srivastava et al., 2014) was included in the larger configurations as a regularisation measure against overfitting. All three configurations reached a validation accuracy of 100% on this small dataset, so the smallest configuration was selected on the reasoning that it was the least likely to overfit while matching the larger networks' validation performance. It was then retrained for 100 epochs on the full SMOTE-balanced training set using Keras (Chollet, 2015) on top of TensorFlow (Abadi et al., 2016), with the Adam optimiser (Kingma and Ba, 2015) and a softmax output layer suited to the three-class problem.

It is worth noting that an earlier from-scratch implementation of a neural network, built using only NumPy to demonstrate the underlying forward-pass and backpropagation mechanics, achieved only 17% test accuracy. On inspection this stemmed from a bug in how the training targets were compared against the network's three output probabilities during backpropagation, rather than from any weakness in the manual approach itself. That version was set aside in favour of the properly tuned Keras model reported here, but it is a useful reminder that a working textbook implementation of backpropagation does not, on its own, guarantee a working model; getting the loss function and label encoding correctly aligned with the output layer matters just as much as the underlying algorithm.

4. Results

Table 1 summarises the test-set performance of the tuned Random Forest and the tuned ANN, evaluated on the same 41-row held-out test split.

Table 1. Test-set performance of the tuned Random Forest and the tuned ANN.

Model	Accuracy	Precision (macro)	Recall (macro)	F1-score (macro)
Random Forest (tuned)	97.6%	0.97	0.93	0.95

ANN (tuned)	95.1%	0.96	0.91	0.93
-------------	-------	------	------	------

Table 2. ROC-AUC (one-vs-rest) for both models.

Model	ROC-AUC (one-vs-rest)
Random Forest (tuned)	1.00
ANN (tuned)	0.99

The Random Forest's confusion matrix showed one High-horsepower car misclassified as Medium, with every Low and Medium car correctly classified. The ANN's confusion matrix showed a broadly similar pattern: one High car misclassified as Medium, and one Medium car misclassified as Low, while every Low car was correctly identified. Both models therefore made a small number of errors concentrated at the boundary between adjacent horsepower categories rather than confusing categories at opposite ends of the scale, which is a reasonably encouraging sign that both are learning a sensible underlying structure rather than guessing.

5. Evaluation

On this test split, the Random Forest edged out the ANN across every metric reported, though the margin is not large: 97.6% versus 95.1% accuracy, and a macro F1-score of 0.95 versus 0.93. Given the test set contains only 41 cars, and only five of them belong to the High category, this gap is well within the range that could shift with a different train-test split, so it would be an overstatement to conclude that the Random Forest is definitively the stronger model for this task rather than simply the model that happened to generalise slightly better on this particular sample.

The two models arrive at their results in quite different ways, and this is where the more useful comparison lies. The Random Forest is well suited to a small, mixed-type tabular dataset like this one: it required very little tuning to perform well, trained in a fraction of a second, and its feature splits can in principle be inspected to understand which variables, such as engine size, curb weight and price, are driving its predictions. The ANN, by contrast, needed considerably more design decisions, including architecture, dropout rate, learning rate and number of epochs, and took noticeably longer to train, without ultimately outperforming the simpler model. This is a fairly common pattern with small tabular datasets: neural networks tend to need larger amounts of data to show a clear advantage over tree-based ensemble methods, and with only 201 rows in total, this dataset is on the small side for a neural network to play to its strengths.

Both models benefited from the SMOTE and tuning steps, though in different ways. The untuned Random Forest baseline scored a broadly similar accuracy (97.6%) but a noticeably lower macro F1-score (0.95 for the untuned baseline's recall specifically dropped on the High

class) than the fully tuned version, indicating that the grid search over `n_estimators`, and more importantly the SMOTE rebalancing, primarily helped the model perform more consistently on the minority High class rather than lifting overall accuracy, which is exactly the outcome imbalance-handling techniques are intended to produce. Without any imbalance handling, a model trained on the original 18:92:50 class split would have had every incentive to under-predict High-horsepower cars, since doing so would barely have dented its overall accuracy score.

6. Conclusion

For this particular dataset and task, the tuned Random Forest performed marginally better than the tuned ANN, and did so with considerably less tuning effort and training time. This does not mean Random Forests are universally preferable to neural networks for tabular classification; rather, it reflects that with a small dataset of only 201 rows and a modest number of features, an ensemble tree method has less to gain from additional model capacity than a neural network does, and reaches a strong result more efficiently. A neural network's advantages would likely become clearer with a substantially larger dataset, additional feature engineering, or a more complex non-linear relationship between features and target than exists here.

Future work on this dataset could explore whether feature engineering, such as combining engine size and weight into a power-to-weight ratio, narrows the gap further, whether a wider hyperparameter search for the ANN (more architectures, more epochs, different optimisers) changes the outcome, and whether the results hold up under repeated k-fold cross-validation rather than a single train-test split, given how few High-horsepower examples exist in this dataset to test against.

References

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G.S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mane, D., Monga, R., Moore, S., Murray, D., Olah, C., Schuster, M., Shlens, J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P., Vanhoucke, V., Vasudevan, V., Viegas, F., Vinyals, O., Warden, P., Wattenberg, M., Wicke, M., Yu, Y. and Zheng, X. (2016) 'TensorFlow: a system for large-scale machine learning', in Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16). Savannah, GA, 2–4 November, pp.265–283.
- Abdel-Hak, A. (2026) Machine Learning and Pattern Recognition. Module lecture notes, B9DA109. Dublin Business School. Unpublished.
- Breiman, L. (2001) 'Random forests', Machine Learning, 45(1), pp.5–32.

- Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) 'SMOTE: synthetic minority over-sampling technique', *Journal of Artificial Intelligence Research*, 16, pp.321–357.
- Chollet, F. (2015) Keras. Available at: <https://keras.io> (Accessed: 22 July 2026).
- Kingma, D.P. and Ba, J. (2015) 'Adam: a method for stochastic optimization', paper presented at the 3rd International Conference on Learning Representations (ICLR), San Diego, CA, 7–9 May.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011) 'Scikit-learn: machine learning in Python', *Journal of Machine Learning Research*, 12, pp.2825–2830.
- Schlimmer, J.C. (1987) Automobile data set. UCI Machine Learning Repository. Available at: <https://archive.ics.uci.edu/ml/datasets/automobile> (Accessed: 22 July 2026).
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014) 'Dropout: a simple way to prevent neural networks from overfitting', *Journal of Machine Learning Research*, 15(1), pp.1929–1958.